

0.93.4. Un jeu de bombes — des flammes plus longues

Le 0.93.3, mais chaque bras de flamme va jusqu'à 2 cases, et s'arrête au premier X : une boucle `for` et `break`.

1. Ce qu'il faut comprendre

C'est le 0.93.3, avec une chose en plus : des bras de deux cases. L'explosion fait maintenant une croix plus grande : jusqu'à 2 cases de chaque côté.

Ce qui est nouveau ici : un bras est une boucle. Pour la droite : `for (uint8_t k = 1; k <= 2; k++)`. `k` vaut 1, puis 2 : la case `fx + 1`, puis la case `fx + 2`. `<=` : « plus petit ou égal », donc 2 compris.

`break` : le mur arrête le bras. Dans la boucle, d'abord : `if (lire(fx + k, fy) == ALPHABET[23]) break;`.

`break` sort de la boucle tout de suite (le 0.33) : les cases suivantes ne sont même pas regardées. La flamme ne traverse pas un pilier.

Déroulons, bombe en (1, 1) : à droite, `k` = 1 : (2, 1) vide, flamme. `k` = 2 : (3, 1) vide, flamme. À gauche, `k` = 1 : (0, 1), le cadre : `break`, rien. En bas, `k` = 1 : (1, 2), flamme. `k` = 2 : (1, 3), flamme. En haut : (1, 0), le cadre : rien.

Et bombe en (2, 1) ? En bas, `k` = 1 : (2, 2), un pilier : `break` tout de suite. Rien en dessous.

Pourquoi `break` protège aussi les nombres : à gauche, `fx - k`. Si `fx` vaut 1, `fx - 2` ne donnerait pas -1 mais 255 (le 0.91.6). Mais (0, 1) est toujours un X : `break` arrive avant `k` = 2. Le cadre garde les calculs dans le terrain.

Le reste ne change pas : `flammes(0)` fait les mêmes boucles, avec les mêmes `break` : elle efface exactement les mêmes cases.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1303 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	87 o	← la console, à l'allumage – la mise en route de la console
chaque_minuteur()	124 o	← main() – une fonction fournie par le langage
caseFeu()	71 o	← flammes() – ta fonction
flammes()	285 o	← main() – ta fonction
main()	440 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte – attend de pouvoir écrire une case
AdresseCase	10 o	← effacer(), texte(), lire(), poser() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
CopierTuiles	67 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	80 o	← CopierTuiles – 5 lettres sur 43
" "	1 o	← texte() – un texte gravé
"-"	1 o	← texte() – un texte gravé

Lettres gravées : BOX-|

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

lire()	x 5	79 o sur place, dans flammes(), main() + AdresseCase (10 o)
poser()	x 6	78 o sur place, dans main() + AdresseCase (10 o)
texte()	x 2	36 o sur place, dans caseFeu() + AdresseCase (10 o), EcrireTexte (10 o)
effacer()	x 2	30 o sur place, dans caseFeu(), main() + AdresseCase (10 o), EffacerCases (12 o)
bouton()	x 5	25 o sur place, dans main() + LireManette (37 o)
image()	x 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	x 1	2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :

caseFeu()	x 5	108 o sur place, dans flammes()
flammes()	x 2	13 o sur place, dans main()
chaque_minuteur()	x 1	10 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UN JEU DE BOMBES ----
// Les lettres : X = un mur, O = le joueur, B = une bombe, W = un ennemi.
// ALPHABET[23] : le X    ALPHABET[14] : le O
// ALPHABET[1]  : le B    ALPHABET[22] : le W

#include <effacer>    // efface des cases, ou tout le fond
#include <texte>      // écrit un texte à l'écran
#include <lire>        // lit la tuile posée sur une case
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <poser>       // pose une tuile sur une case du fond
#include <chaque>      // répond 1 toutes les n millisecondes
#include <bouton>      // lit un bouton de la manette
```

2. UNE VARIABLE — OX

```
uint8_t ox = 1;          // le O, le joueur : sa colonne... (le 0.93.1)
```

3. UNE VARIABLE — OY

```
uint8_t oy = 1;          // ...et sa ligne. Il part du coin (1, 1).
```

4. UNE VARIABLE — NX

```
uint8_t nx = 0;          // la case d'ARRIVÉE, calculée AVANT de bouger (le 0.82)
```

5. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

6. UNE VARIABLE — BOMBE

```
uint8_t bombe = 0;          // 1 : une bombe est posée ; 0 : pas de bombe (le 0.93.2)
```

7. UNE VARIABLE — BX

```
uint8_t bx = 0;            // la place de la bombe : sa colonne...
```

8. UNE VARIABLE — BY

```
uint8_t by = 0;            // ...et sa ligne
```

9. UNE VARIABLE — BDELAI

```
uint8_t bdelai = 0;        // les images passées depuis la pose de la bombe (le 0.93.3)
```

10. UNE VARIABLE — FEU

```
uint8_t feu = 0;           // 1 : les flammes sont à l'écran ; 0 : non
```

11. UNE VARIABLE — FDELAI

```
uint8_t fdelai = 0;        // les images passées depuis l'explosion
```

12. UNE VARIABLE — FX

```
uint8_t fx = 0;            // le centre de l'explosion : sa colonne...
```

13. UNE VARIABLE — FY

```
uint8_t fy = 0;            // ...et sa ligne

// ---- UNE case de flamme (le 0.93.3) ----
// allume = 1 : on dessine la flamme ; allume = 0 : on l'efface.
// debout = 1 : une flamme verticale, le | ; debout = 0 : horizontale, le -.
```

14. UNE FONCTION — CASEFEU()

```
void caseFeu(uint8_t c, uint8_t l, uint8_t allume, uint8_t debout) {
    if (allume == 0) {
        effacer(c, l, 1);          // la flamme s'éteint : la case est vide
    } else {
        if (debout == 1) {
            texte(c, l, "|");
        } else {
            texte(c, l, "-");
        }
    }
}

// ---- Toute l'explosion (le 0.93.3) : NOUVEAU : des bras de DEUX cases ----
//
//      |
//      |          chaque bras : jusqu'à 2 cases,
//  - - - -      mais il s'arrête au premier X.
//      |          break : on sort de la boucle, tout de suite.
//      |
```

15. UNE FONCTION — FLAMMES()

```
void flammes(uint8_t allume) {
    caseFeu(fx, fy, allume, 0);           // le centre
    for (uint8_t k = 1; k <= 2; k++) {    // à droite : fx + 1, puis fx + 2
        if (lire(fx + k, fy) == ALPHABET[23]) break; // un X : le bras s'arrête là
        caseFeu(fx + k, fy, allume, 0);
    }
    for (uint8_t k = 1; k <= 2; k++) {    // à gauche : fx - 1, puis fx - 2
        if (lire(fx - k, fy) == ALPHABET[23]) break;
        caseFeu(fx - k, fy, allume, 0);
    }
    for (uint8_t k = 1; k <= 2; k++) {    // en bas
        if (lire(fx, fy + k) == ALPHABET[23]) break;
        caseFeu(fx, fy + k, allume, 1);
    }
    for (uint8_t k = 1; k <= 2; k++) {    // en haut
        if (lire(fx, fy - k) == ALPHABET[23]) break;
        caseFeu(fx, fy - k, allume, 1);
    }
}
```

16. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- Le terrain : le cadre et les piliers X (le 0.93) ----
    for (uint8_t l = 0; l < 17; l++) {
        for (uint8_t c = 0; c < 19; c++) {
            if (l == 0 || l == 16 || c == 0 || c == 18) {    // le cadre
                poser(c, l, ALPHABET[23]);
            }
            if (c % 2 == 0 && l % 2 == 0) {                  // les piliers : colonne ET ligne paires
                poser(c, l, ALPHABET[23]);
            }
        }
    }

    poser(ox, oy, ALPHABET[14]);                          // le 0 à sa place (le 0.93)

    while (true) {
        image();

        // ---- le 0 bouge avec la croix, un pas toutes les 150 ms (le 0.93.1) ----
        if (chaque(150)) {
            // 1. la case d'ARRIVÉE : on part de la case actuelle (le 0.82).
            //     else if : une seule flèche à la fois, pas de pas en diagonale.
            nx = ox;
            ny = oy;
            if (bouton(DROITE)) {
                nx = ox + 1;
            } else if (bouton(GAUCHE)) {
                nx = ox - 1;
            } else if (bouton(BAS)) {
                ny = oy + 1;
            } else if (bouton(HAUT)) {
                ny = oy - 1;
            }

            // 2. une flèche est tenue (l'arrivée n'est pas la case où l'on est)...
            if (nx != ox || ny != oy) {
                // 3. ...et l'arrivée est VIDE : ni X, ni B, ni flamme. (le 0.93.2)
                //     lire() rend 0 pour une case vide : c'est l'espace, la tuile 0.
                if (lire(nx, ny) == 0) {
                    effacer(ox, oy, 1);                    // le 0 quitte sa case...
                    if (bombe == 1 && ox == bx && oy == by) {
                        poser(bx, by, ALPHABET[1]);        // ...mais s'il était sur la bombe, la B apparaît (le 0.93.2)
                    }
                    ox = nx;                                // ...et va sur la case d'arrivée
                    oy = ny;
                    poser(ox, oy, ALPHABET[14]);
                }
            }
        }

        // ---- A : poser une bombe, là où est le 0 (le 0.93.2) ----
        // bombe == 0 : une seule bombe à la fois.
        // feu == 0 : pas pendant les flammes (le 0.93.3).
        if (bouton(A) && bombe == 0 && feu == 0) {
            bombe = 1;
            bx = ox;                                        // la bombe est sous le 0
            by = oy;
            bdelai = 0;                                    // on commence à compter (le 0.93.3)
        }

        // ---- la bombe explose au bout de 2 secondes (le 0.93.3) ----
        // image() revient 60 fois par seconde : 120 images, 2 secondes.
        if (bombe == 1) {
            bdelai = bdelai + 1;
        }
    }
}
```

```

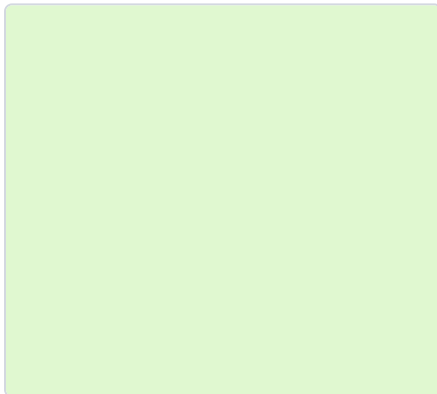
    if (bdelai == 120) {
        bombe = 0;                                // plus de bombe...
        fx = bx;                                  // ...elle explose, là où elle était
        fy = by;
        feu = 1;
        fdelai = 0;
        flammes(1);                                // on dessine les flammes
    }
}

// ---- les flammes s'éteignent au bout d'une demi-seconde (30 images) (1e 0.93.3) ----
if (feu == 1) {
    fdelai = fdelai + 1;
    if (fdelai == 30) {
        feu = 0;
        flammes(0);                                // on efface les MÊMES cases
        poser(ox, oy, ALPHABET[14]);                // le 0 était peut-être dans une flamme : on le remet
    }
}
}
}
}

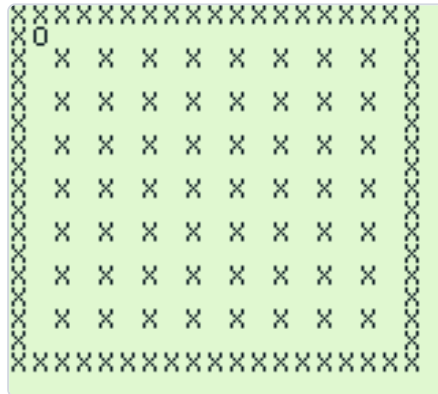
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



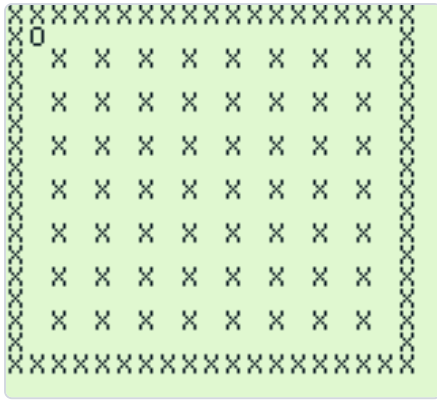
la toute première image — la console vient de démarrer — image 1



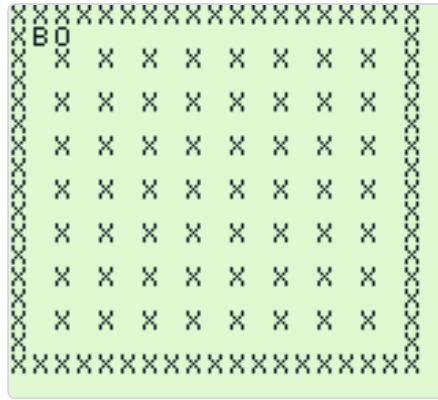
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

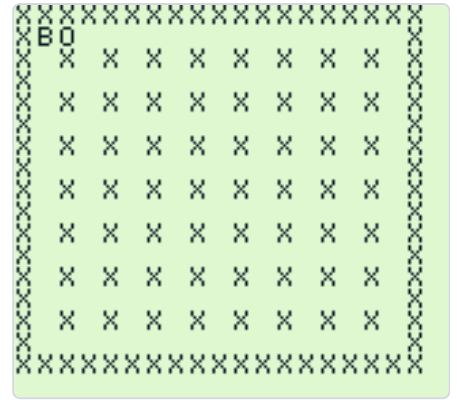
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



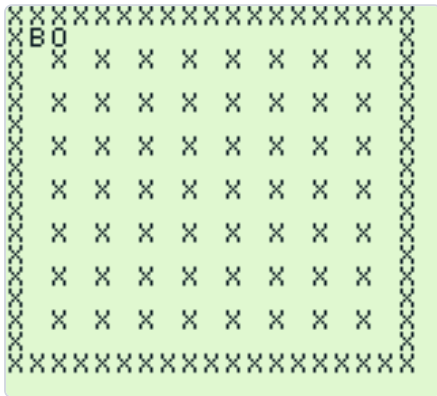
A — l'écran n'a pas bougé



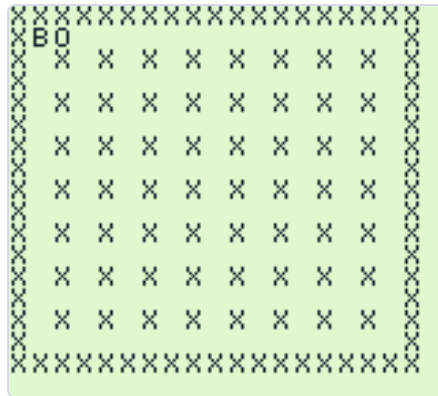
DROITE — l'écran a changé



GAUCHE — l'écran n'a pas bougé



HAUT — l'écran n'a pas bougé



BAS — l'écran n'a pas bougé

5. Ce qu'on doit voir

L'explosion fait une croix de 2 cases de chaque côté ; un X arrête le bras.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ à droite : 2 cases, - - -, puis rien
- ✓ en bas : | en (1, 2) et (1, 3), puis rien
- ✓ le cadre arrête le bras de gauche et du haut
- ✓ le O, à 4 cases, n'est pas touché

Ensuite : **0.93.5. Un jeu de bombes — un ennemi W** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++